National University of Singapore

DSA5208 Scalable Distributed Computing for Data Science

Project 3 Report

Parallel PageRank with MPI and OpenMP

Authors: Yan Tianao, Wan Gehong, Wang Tianyi

Student IDs: A0354280H, A0356324E, A0357264X

Date: November 2026

Abstract

This project presents C++17 Serial, MPI-only, and MPI+OpenMP PageRank using a shared Compressed Sparse Row (CSR) representation. Four real graphs are evaluated on the NUS Vanda cluster with five measured runs per configuration. A comprehensive 1-128 CPU sweep tests every feasible rank/thread layout and obtains best Hybrid speedups of 6.32x-12.20x over same-sweep cluster Serial C++; cross-node speedups from separate job series are not compared. Communication reaches 93.0%-99.4% of MPI-only solve time at 128 CPUs, explaining strong-scaling saturation. In-edge-work partitioning improves seven of eight 32-CPU comparisons. Demand-driven score exchange improves 15 of 16 controlled comparisons, reaching 5.260x over full-vector exchange; in the only regression, solve time increases by 17.6% (full/demand speedup 0.850x). NetworkX validates complete web-Google vectors from Serial and demand-driven MPI runs with L1 errors near 1.2×10^{-12}.

Contents

1. Introduction 1

2. Implementation 1

2.1 Shared graph representation 1

2.2 Serial baseline 1

2.3 MPI-only design 2

2.4 Hybrid MPI+OpenMP design 2

2.5 In-edge-work partition 2

2.6 Demand-driven score exchange 2

2.7 Instrumentation 3

3. Experimental Method 3

3.1 Data sets 3

3.2 Platforms and compilation 3

3.3 Measurement protocol 3

4. Correctness Validation 5

5. Performance Results 5

5.1 Experiment 1: Local Mac results 5

5.2 Experiment 2: Fixed 32-CPU rank/thread decomposition 5

5.3 Experiment 3: Strong scaling 6

5.4 Experiment 4: Weak and structural scaling 8

5.5 Experiment 5: In-edge-work partitioning 9

5.6 Experiment 6: Phase timing and the scaling limit 10

5.7 Experiment 7: Demand-driven score exchange 12

6. Discussion 13

6.1 Effects of graph structure 13

6.2 Design trade-offs 13

6.3 End-to-end optimization pipeline 14

6.4 Threats to validity 14

7. Conclusion 15

AI Use Declaration 15

References 15

1. Introduction

PageRank assigns a stationary importance score to every vertex in a directed graph. It was introduced for ranking web pages but is also a useful parallel-computing case study because each iteration combines irregular sparse computation with global synchronization [1]. Local updates depend on scores owned by other processes, while dangling vertices and the convergence criterion require global reductions.

For a graph $G=(V,E)$, let $N=|V|$ and let $\mathrm{outdeg}(u)$ be the out-degree of vertex u. With damping factor $\alpha=0.85$, uniform initialization, and uniform redistribution of dangling mass, this project computes

$$r_{\mathrm{new}}(v)=\frac{1-\alpha}{N}+\alpha\frac{D}{N}+\alpha\sum_{u\to v}\frac{r(u)}{\mathrm{outdeg}(u)}.$$

where the sum is over all incoming edges $u\to v$ and D is the total score on vertices with zero out-degree. Iteration stops when the global L1 difference satisfies

$$\sum_{v}|r_{\mathrm{new}}(v)-r(v)|<10^{-12},$$

or after 1,000 iterations. Scores are stored in double precision.

The project addresses five main questions:

- How do MPI-only and Hybrid PageRank behave under strong scaling and controlled replica weak scaling?
- At a fixed CPU count, how should work be divided between MPI ranks and OpenMP threads?
- Does balancing estimated incoming-edge work improve runtime?
- Which iterative phases limit scaling at high process counts?
- Can demand-driven exchange reduce score communication without introducing greater planning, packing, or synchronization costs?

The report evaluates a shared-CSR implementation in Serial, MPI-only, and Hybrid modes. The completed experiments cover local workflow validation, fixed-CPU rank/thread decomposition, strong scaling, BerkStan replica weak scaling, UK induced-subgraph structural scaling, in-edge-work partitioning, per-phase timing, and a controlled comparison of full-vector and demand-driven score exchange. All variants use the same PageRank equation and convergence rule; the exchange experiment holds the partition and rank/thread layout fixed to isolate communication design.

2. Implementation

2.1 Shared graph representation

All implementations use one incoming Compressed Sparse Row (CSR) representation. For each target vertex, a contiguous range stores its incoming source vertices. The out-degree array contains one value for every vertex, computed from the complete deduplicated input graph. Input edges are deduplicated, self-loops are retained, declared isolated vertices are preserved, and original IDs are mapped to dense IDs. Large inputs are converted to a binary `.prcsr` format to avoid repeated text parsing.

The pull formulation is convenient for ownership: the rank that owns target vertex v stores all incoming edges of v and is solely responsible for computing its new score. This avoids conflicting writes and produces a deterministic ownership rule across serial, MPI, and hybrid modes.

2.2 Serial baseline

The serial version holds the complete incoming CSR, out-degree array, old score vector, and new score vector. In every iteration it computes dangling mass, updates every target, calculates the L1 difference, and swaps the score buffers. File loading and validation are outside the reported solve interval.

2.3 MPI-only design

Rank 0 loads and validates the graph. Target vertices are divided into contiguous blocks and the corresponding incoming CSR slices are distributed. Each rank stores:

- its local target range and incoming edges;
- a complete global out-degree array, broadcast from rank 0;
- a complete old score vector;
- a local new score vector.

The replicated out-degree array is indexed by original dense source-vertex ID. Each value is the source vertex's out-degree in the complete deduplicated input graph, not an out-degree recomputed from the rank's local incoming-edge slice.

An iteration performs the following operations:

1. Each rank sums dangling scores owned by its local target range. 2. `MPI_Allreduce` forms the global dangling mass. 3. Each rank updates its local targets using its incoming CSR slice. 4. Each rank computes a local L1 difference and `MPI_Allreduce` forms the global difference. 5. `MPI_Allgatherv` assembles all local new scores into the next complete score vector.

The complete score and out-degree vectors simplify random source access and validation, but each rank stores $O(N)$ scores and $O(N)$ out-degrees. `MPI_Allgatherv` communicates the full score vector every iteration. This replicated storage and full-vector communication are the central scalability limitations of the baseline.

2.4 Hybrid MPI+OpenMP design

The hybrid version keeps the MPI data distribution and collective sequence unchanged. OpenMP parallelizes local dangling summation, target updates, and L1 difference calculation. MPI is initialized with `MPI_THREAD_FUNNELED`; collective calls occur outside OpenMP parallel regions. CPU affinity is controlled by the cluster launch scripts, and each target is updated by exactly one thread.

Reducing the number of MPI ranks has two possible benefits: fewer collective participants and fewer replicated process-level data structures. The cost is that local work must scale efficiently across OpenMP threads. The fixed-32-CPU experiment tests this trade-off directly.

2.5 In-edge-work partition

The default equal-vertex partition gives ranks similar numbers of target vertices. This can be imbalanced because update cost depends mainly on in-degree. The in-edge-work partition instead uses the weight

$$w(v) = 1 + \mathrm{indeg}(v).$$

and creates contiguous blocks with approximately equal total weight. The constant term accounts for per-vertex loop overhead. Only ownership boundaries change: the PageRank formula, collectives, score replication, OpenMP loops, and convergence test remain unchanged. The corresponding experiment therefore isolates the effect of computational load balance.

2.6 Demand-driven score exchange

The optimized distributed variant replaces the replicated score vector and per-iteration `MPI_Allgatherv` with a fixed demand plan. After CSR distribution, each rank scans its local incoming sources, removes locally owned vertices, and sorts and deduplicates the remaining remote source IDs. Because ownership ranges are contiguous, the IDs are grouped by owner. An initial `MPI_Alltoall` exchanges request counts and `MPI_Alltoallv` sends the requested vertex IDs to their owners. Each incoming edge is then mapped once to either a local-score index or a slot in the compact remote-score array.

Each rank stores two local score buffers, the deduplicated remote scores needed by its targets, the scores requested from it by other ranks, and the fixed edge-to-score slot map. The complete out-degree array

remains replicated because every pull contribution requires the source vertex's out-degree in the original complete graph. In each iteration, a rank packs the current local scores requested by its peers, exchanges these values with `MPI_Alltoallv`, computes local updates from local and received scores, and uses the same two `MPI_Allreduce` operations for dangling mass and global L1 convergence. A complete score vector is gathered to rank 0 only after the timed solve when output or validation requests it.

Dependency construction is outside solve time. Packing and iterative value exchange are inside solve time and are timed separately. Uniform initial scores are created locally, so there is no separate initial score broadcast; graph distribution and dependency construction are reported independently.

2.7 Instrumentation

The program records load time, distribution time, solve time, convergence information, residual, score-sum error, and maximum per-rank peak resident memory. Partitioning experiments additionally record static per-rank vertex, edge, and work statistics and measured update-time imbalance. The phase-timing metrics record rank-mean and rank-maximum time for local computation, `MPI_Allreduce`, and score exchange. For demand exchange, dependency-build time, packing time, unique remote dependencies, communication partners, score values and bytes per iteration, and exchange-related state bytes are also recorded. The payload-volume metric counts score values and excludes MPI protocol overhead. Phase timers add no per-phase barrier; statistics are reduced only after convergence.

3. Experimental Method

3.1 Data sets

All four inputs are directed hyperlink graphs: a vertex represents a web or Wikipedia page, and an edge $u \to v$ represents a hyperlink from page u to page v. web-Google was released for the 2002 Google Programming Contest. web-BerkStan contains pages from the `berkeley.edu` and `stanford.edu` domains collected in 2002. enwiki-2013 is a 2013 snapshot of English Wikipedia. uk-2002 was obtained by a 2002 UbiCrawler crawl of the `.uk` domain [6,7]. Their processed sizes are shown below.

Table 1: Graph data sets and processed CSR sizes.

Data set	Vertices	Edges	Binary CSR size
web-Google	875,713	5,105,039	0.035 GiB
web-BerkStan	685,230	7,600,595	0.041 GiB
enwiki-2013	4,203,323	101,311,614	0.456 GiB
uk-2002	18,520,486	298,113,762	1.456 GiB

The first three data sets are distributed through SNAP, while uk-2002 is distributed in LAW/WebGraph format. Checksums and conversion metadata are stored with the project. The graphs differ substantially in both size and edge density, which helps distinguish computation-heavy behavior from communication-dominated behavior.

3.2 Platforms and compilation

Local workflow tests used an Apple M3 Pro Mac with 11 CPU cores and 18 GiB of memory. The reported scaling experiments used Vanda compute nodes with 32 allocated CPUs per node. Runs up to 32 CPUs used one node, 64 CPUs used two nodes, and 128 CPUs used four nodes. The code was compiled as C++17 with `-O3`, Open MPI 4.1.8, and OpenMP enabled.

3.3 Measurement protocol

Each local and final Vanda configuration used one unrecorded warm-up followed by five measured runs. Tables and plots use the median, with the first and third quartiles retained for uncertainty bands. The timed solve includes iteration computation, collectives, and convergence handling, but excludes graph loading, initial distribution, output, and optional serial verification.

Vanda records were collected in separate PBS job series, so serial timings from different series are not interchangeable. The fixed-32 layout jobs ran on IN-006 and did not measure Serial. The comprehensive single-node strong-scaling jobs ran Serial and every 2-32 CPU layout sequentially within the same per-graph PBS job on IN-015; consequently, their Serial-to-1x32 speedups and 1x32-to-32x1 comparisons control node identity. The UK waterfall ran all five stages within one IN-002 job. The earlier strong-scaling serial measurements are retained only as provenance and for the same-node UK consistency check in Section 6.3; they are not used to calculate the comprehensive strong-scaling speedups.

Table 2: Serial-baseline provenance and five-run median solve times (s).

Job series	Node	web-Google	web-BerkStan	enwiki-2013	uk-2002
Earlier strong scaling	IN-001 (Google); IN-002 (others)	5.357	2.088	34.499	80.802
Fixed-32 layout decomposition	IN-006	not measured	not measured	not measured	not measured
Comprehensive strong scaling	IN-015	3.379	2.005	20.641	61.542
Controlled UK waterfall	IN-002	-	-	-	78.663

The 30%-40% shifts for enwiki and UK exceed ordinary within-series variation and coincide with different node assignments and collection times. The records do not identify a single hardware or system-state cause, so the report treats this as a batch-placement effect and avoids cross-series speedup ratios.

For total CPU count C, speedup and efficiency are

$$S(C) = \frac{T_{\text{serial}}}{T_{\text{parallel}}(C)}, \qquad E(C) = \frac{S(C)}{C}.$$

The serial denominator is the C++ serial solve from the same comprehensive Vanda sweep and graph. Strong scaling keeps the input fixed while increasing CPUs. At each CPU count, the supplementary sweep tests every feasible factorization into MPI ranks and OpenMP threads, with at most 32 threads per rank. Thus, 1-32 CPUs use one node and respectively test 1, 2, 3, 4, 5, and 6 layouts; 64 and 128 CPUs each test six layouts on two and four nodes. All layouts for one graph and resource scope run in the same PBS job. The replica weak-scaling family increases both BerkStan replicas and CPUs by factors 1, 2, 4, 8, 16, and 32.

The UK structural-scaling family uses nested uniform vertex samples of uk-2002, generated with seed 5208 at fractions 1/32, 1/16, 1/8, 1/4, 1/2, and 1/1. Each sampled graph is induced: an original edge is retained only when both endpoints are selected, and out-degrees are recomputed within the retained graph. CPU count increases from 4 to 128 so that vertices per CPU remain nearly constant, while edges per CPU rise from approximately 0.118 million to 2.329 million. This structural-scaling series measures how MPI-only and Hybrid execution respond jointly to increasing resources, induced-graph density, and changing topology.

The demand-exchange comparison uses all four complete graphs at 32 and 64 CPUs. At each CPU count it tests MPI-only and eight-thread Hybrid layouts: 32x1 and 4x8 on one node, and 64x1 and 8x8 on two nodes. Full-vector and demand-driven variants use the same in-edge-work partition, graph, CPU allocation, rank/thread layout, damping factor, tolerance, iteration limit, affinity, and convergence rule. Each pair follows the same one-warm-up plus five-measurement protocol. This design holds load balancing constant and isolates the score-exchange method.

4. Correctness Validation

Correctness was checked at three levels. First, 16 deterministic C++ tests cover a graph with a known solution, all-dangling vertices, duplicate-edge removal, a self-loop, header-declared isolated vertices, rank-sum preservation, residuals, and convergence. Second, explicit consistency checks compare MPI and Hybrid results with the Serial CSR solution on deterministic toy, dangling-vertex, and isolated-vertex graphs. They cover full-vector and demand-driven exchange, both equal-vertex and in-edge-work partitions, one-rank/four-thread Hybrid execution, and a demand configuration with eight ranks for four vertices. All tests pass. These explicit Serial/parallel comparisons cover the test graphs; the formal Vanda jobs do not run the optional serial reference calculation.

Third, web-Google was independently reconstructed in NetworkX 3.7 using the original compressed SNAP edge list and compared with both the complete Serial C++ vector and a complete vector gathered after a four-rank demand-driven MPI run with in-edge-work partitioning. The reference used the same α, uniform personalization and dangling distribution, and a global reference tolerance of 10^{-13}. NetworkX stops at $N \cdot \mathrm{tol}$, so its per-node tolerance was scaled accordingly [4]. Dense internal IDs were aligned with sorted original IDs before comparing complete vectors.

Table 3: Independent NetworkX validation on web-Google.

Validation quantity	Result
Vertices	875,713
Edges	5,105,039
Serial full-vector L1 error	1.198×10^{-12}
Demand MPI full-vector L1 error	1.178×10^{-12}
Demand MPI maximum element error	7.322e-15
Demand MPI rank-sum error	6.661×10^{-15}
Decision	Accepted

Every formal Vanda run records convergence, a one-step residual, and score-sum error. All 1,625 measured runs converged; the maximum residual was 8.324×10^{-13} against a tolerance of 10^{-12}, and the maximum score-sum error was 3.925×10^{-11}.

5. Performance Results

5.1 Experiment 1: Local Mac results

The repeated local experiment contains 60 measured runs: four graphs, three implementations, and five repetitions after one warm-up. The earlier 12-row single-run file is retained for provenance but excluded from this analysis. All repeated runs converged. Median Serial, MPI-only, and 2x2 Hybrid solve times are respectively 1.644 s, 2.899 s, and 1.534 s on Google; 0.894 s, 0.849 s, and 0.597 s on BerkStan; 24.717 s, 19.012 s, and 16.225 s on enwiki; and 37.192 s, 27.642 s, and 24.699 s on UK. Thus, four MPI ranks are slower than Serial on Google, while Hybrid improves the median on every graph. These workstation measurements verify the workflow under repetition; the cluster experiments provide the multi-node evidence used for scaling conclusions.

5.2 Experiment 2: Fixed 32-CPU rank/thread decomposition

At 32 CPUs on one Vanda node, hybrid configurations were faster than 32 MPI ranks for every graph. Bold entries mark the minimum median solve time in each graph column within this fixed-CPU job series. The best configuration used one rank with 32 threads for Google, BerkStan, and enwiki, and two ranks with 16 threads for UK-2002. Because these IN-006 jobs did not include a Serial run, this experiment compares rank/thread decompositions only and reports no speedup over Serial.

Table 4: Fixed-32-CPU layout median solve times (s).

Layout	web-Google	web-BerkStan	enwiki-2013	uk-2002
1x32	**0.287**	**0.331**	**2.000**	8.038
2x16	0.331	0.341	2.402	**7.972**
4x8	0.470	0.395	3.206	10.075
8x4	0.562	0.459	3.409	10.478
16x2	0.675	0.519	3.850	12.198
32x1	0.866	0.629	4.159	13.675

Fewer ranks reduce collective participation and process replication. UK-2002 slightly preferred two ranks, showing that rank/thread layout should be matched to graph structure even within a node.

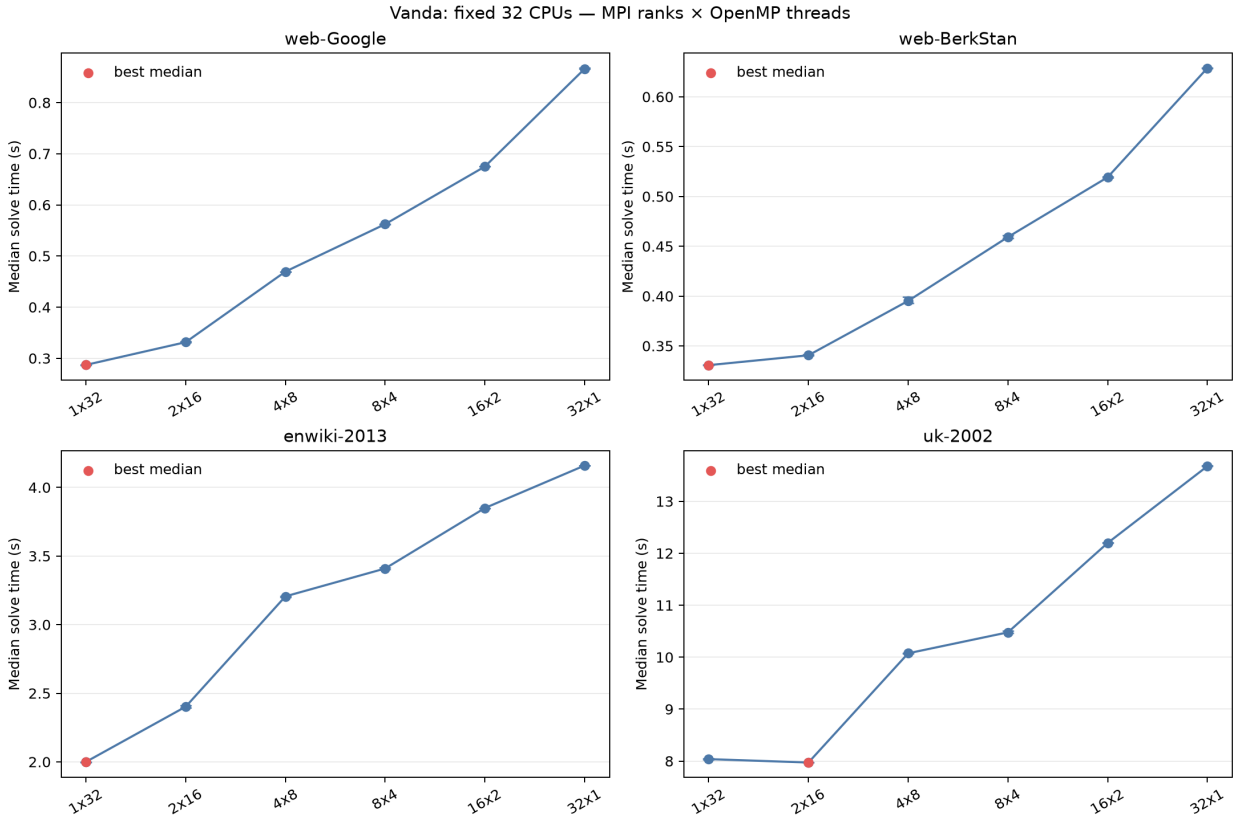


Figure 1: 32-CPU decomposition on Vanda.

5.3 Experiment 3: Strong scaling

The supplementary strong-scaling experiment contains 660 measured runs: 33 configurations per graph, four graphs, and five repetitions. At every CPU count from 1 to 128, it covers every feasible rank/thread factorization with at most 32 threads per rank. The table reports the global minimum selected only after this complete layout sweep.

Table 5: Best points from the comprehensive strong-scaling sweep.

Data set	Serial median	Best layout	CPUs	Best median	Speedup	Efficiency
web-Google	3.379 s	1x32 hybrid	32	0.277 s	12.20x	38.1%
web-BerkStan	2.005 s	1x32 hybrid	32	0.317 s	6.32x	19.8%
enwiki-2013	20.641 s	1x32 hybrid	32	1.989 s	10.38x	32.4%
uk-2002	61.542 s	1x32 hybrid	32	7.320 s	8.41x	26.3%

All four graphs reach their full-vector minimum at 32 CPUs with one MPI rank and 32 OpenMP threads. At 64 CPUs, the best layout is 2x32 for every graph; median times increase to 0.365 s, 0.399 s, 2.693 s, and 8.296 s for Google, BerkStan, enwiki, and UK, respectively. The complete sweep therefore confirms that the additional node and collective coordination outweigh the extra cores after 32 CPUs.

At 128 CPUs, 4x32 is best for Google, BerkStan, and enwiki, with medians of 0.456 s, 0.445 s, and 2.932 s. UK instead selects 16x8 at 8.677 s, showing that graph structure can shift the best layout at high scale even though the best global point remains at 32 CPUs. The expanded layout coverage establishes the 1x32 configurations at 32 CPUs as the complete-sweep minima for enwiki and UK. The UK fixed-CPU experiment in Section 5.2 was a separate job series and placed 2x16 only 0.066 s (0.8%) ahead of 1x32; the comprehensive sweep reverses that near-tie. Strong-scaling minima and speedups are therefore calculated solely from the internally controlled comprehensive sweep rather than mixed across job series.

Baseline choice materially affects the numerical ratio. For example, combining the earlier Google Serial median (5.357 s) with the separate fixed-32 1x32 result (0.287 s) would give 18.66x, but those measurements came from different jobs and nodes. The reported 12.20x uses 3.379 s and 0.277 s from the same IN-015 comprehensive job. The abstract's 6.32x-12.20x range refers only to these controlled same-sweep ratios.

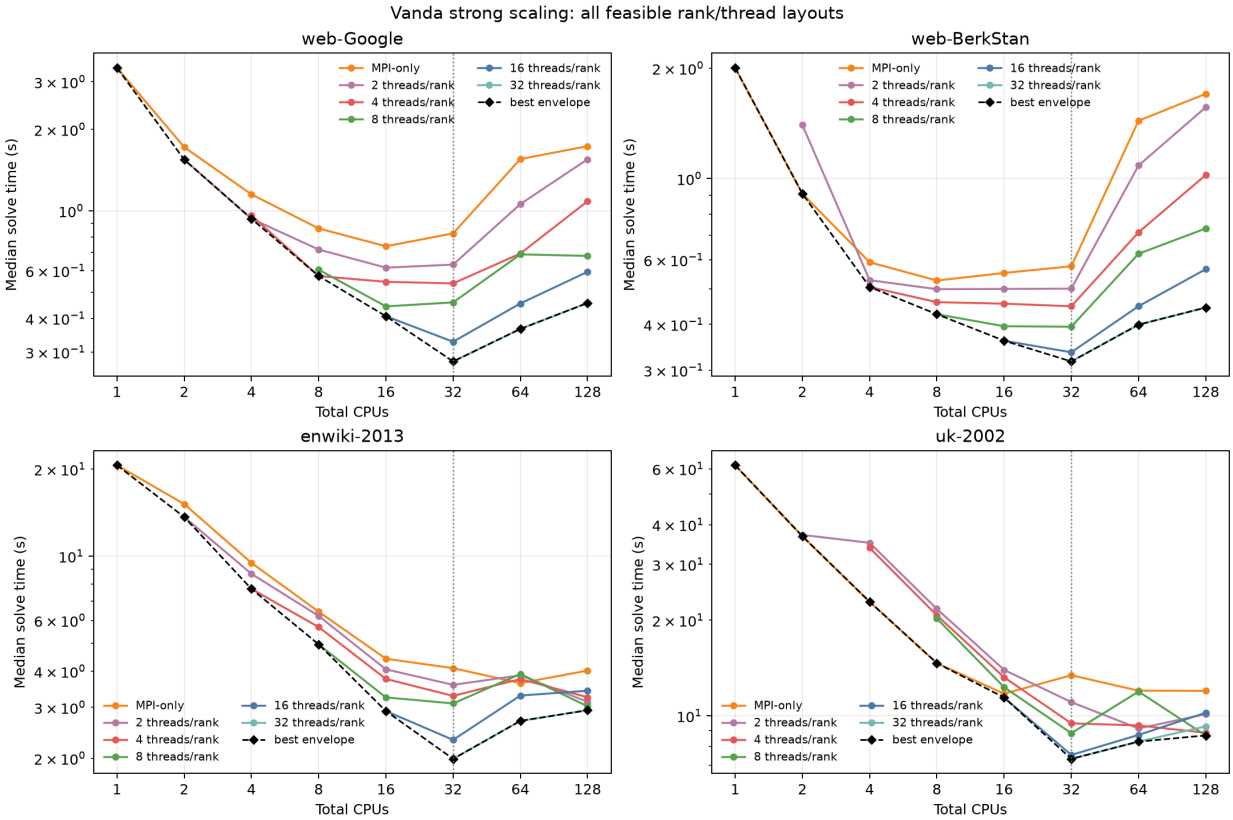


Figure 2: Strong-scaling solve time (five-run medians).

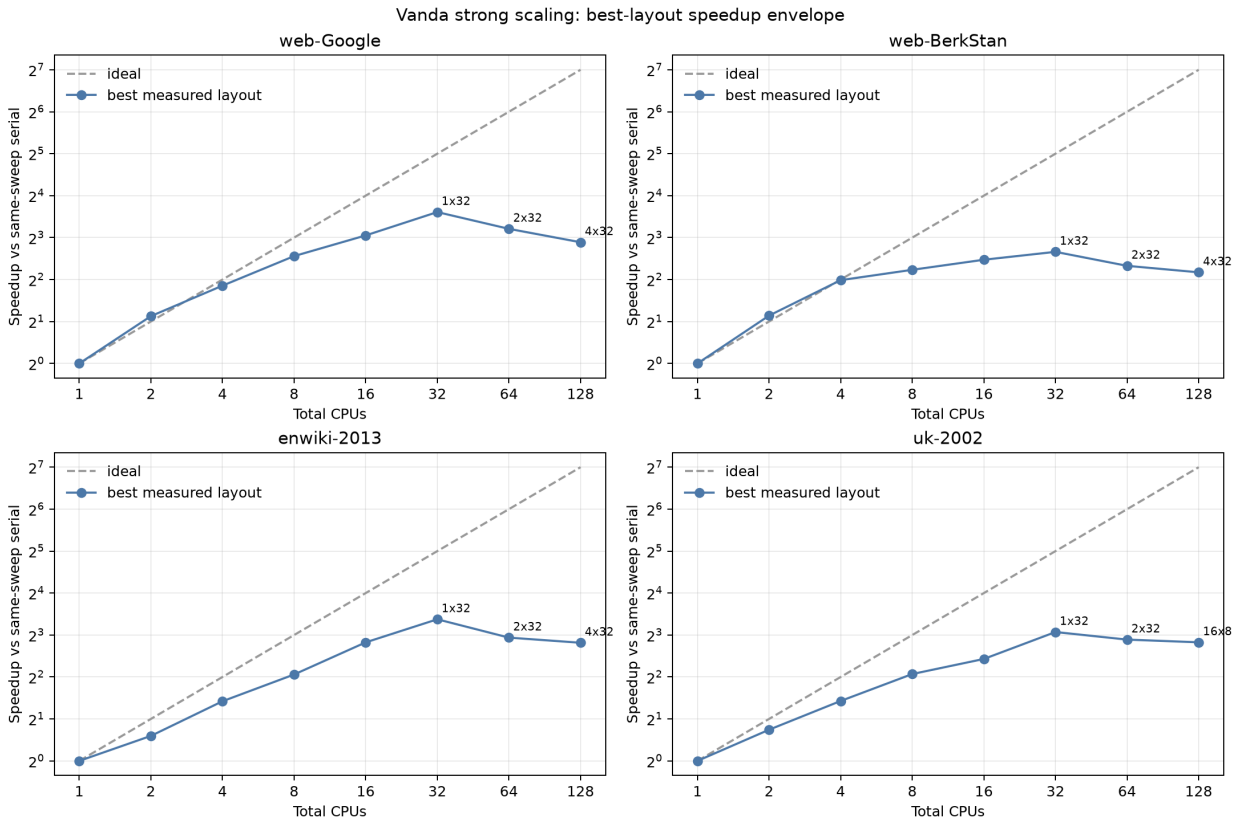


Figure 3: Strong-scaling speedup over Vanda Serial.

5.4 Experiment 4: Weak and structural scaling

The BerkStan replica family keeps both vertices per CPU (171,307.5) and edges per CPU (1.900 million) exactly constant. Ideal weak scaling would therefore keep runtime constant. The measured MPI-only time rises from 1.021 s at 4 CPUs to 13.559 s at 128 CPUs, while hybrid rises from 0.897 s to 10.042 s. At the largest scale, relative weak efficiencies are approximately 7.5% and 8.9%.

This behavior is expected for the implemented communication pattern. The full score vector grows with the replicated graph, and every rank receives it every iteration. Thus, local edge work remains fixed but `Allgatherv` volume, collective coordination, and cross-node latency increase. Replication also preserves the 148-iteration count, ruling out convergence changes as the cause.

The UK induced-subgraph family increases edge work from 0.118 million edges per CPU at 1/32 to 2.329 million at the full graph, while iterations change from 134 to values near 141-143. Its runtime trend therefore captures denser induced subgraphs, evolving graph structure, and communication together. We report this family as structural scaling, providing a realistic complement to the controlled BerkStan replica experiment.

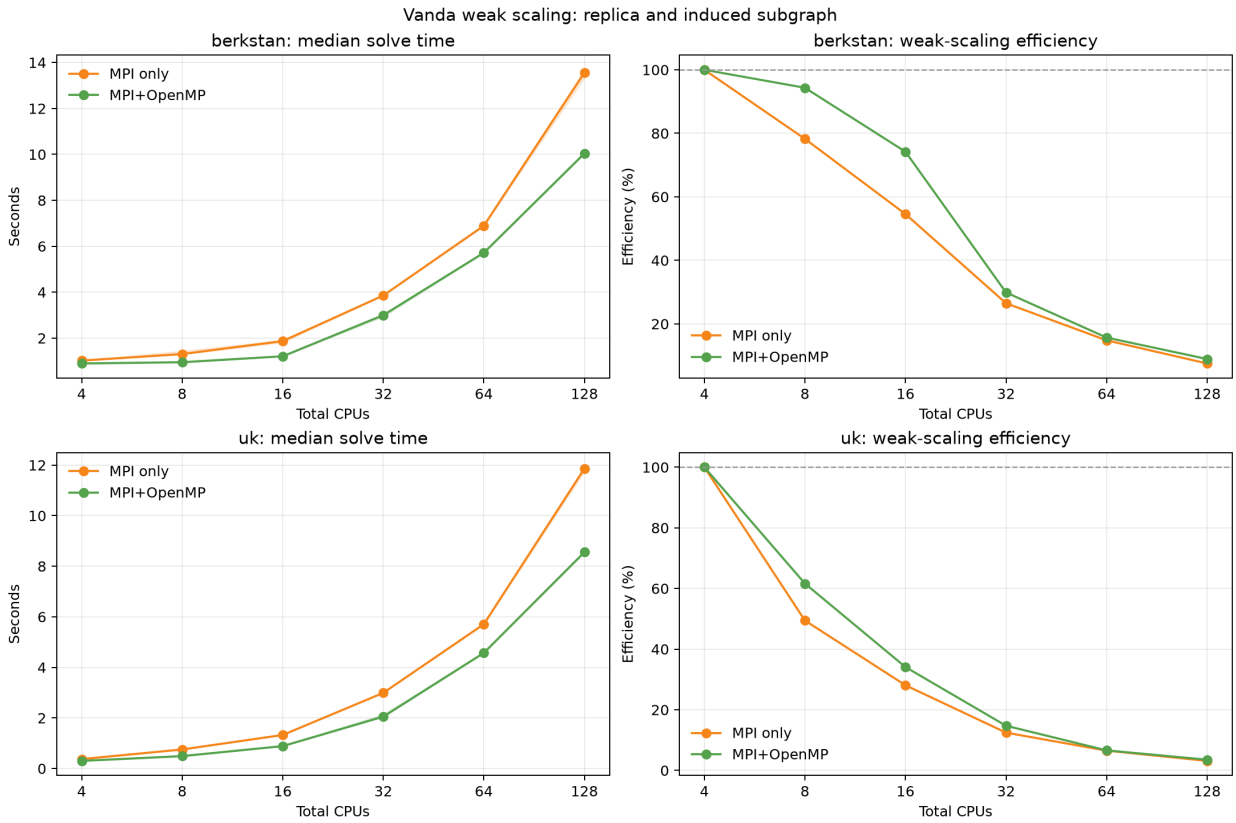


Figure 4: Replica and induced-subgraph scaling.

5.5 Experiment 5: In-edge-work partitioning

Experiment 5 used one 32-CPU node and compared equal-vertex and in-edge-work partitioning under 32x1 MPI and 4x8 Hybrid layouts. All 80 measured runs converged. In-edge-work partitioning reduced both static work imbalance and measured update-time imbalance in every data-set/layout pair. It improved total runtime in seven of eight comparisons.

Table 6: In-edge-work partitioning speedup at 32 CPUs.

Data set	In-edge-work speedup, 32x1	In-edge-work speedup, 4x8
web-Google	1.071x	0.989x
web-BerkStan	1.159x	1.174x
enwiki-2013	1.297x	1.058x
uk-2002	1.044x	1.025x

The largest static correction was BerkStan 32x1, where maximum/mean assigned work fell from 2.924 to 1.072. The largest runtime gain was enwiki 32x1: median solve time fell from 4.155 s to 3.203 s, a 1.297x improvement. Seven of eight configurations improve; Google 4x8 changes by approximately -1.1%, indicating that a small load-balance gain can be comparable to partition-dependent cache effects or measurement variation when communication is unchanged.

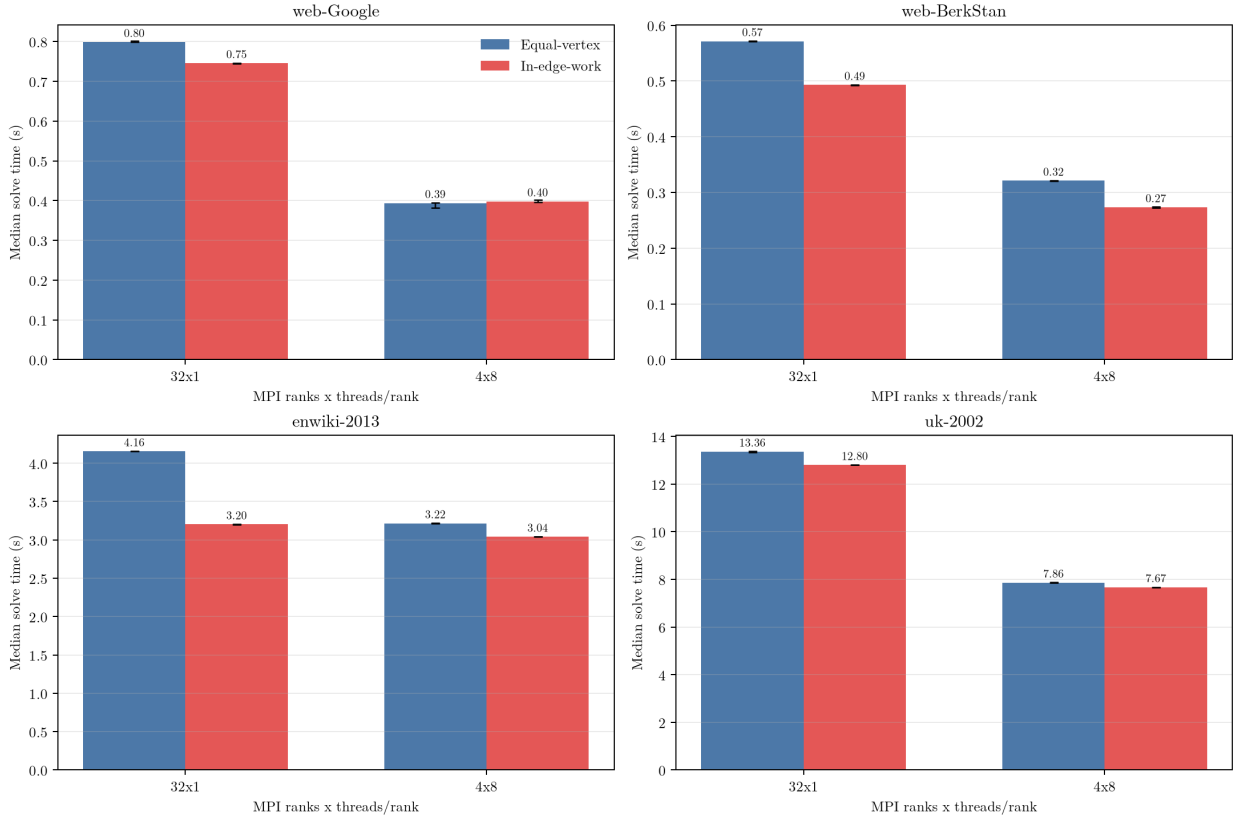


Figure 5: Runtime under vertex- and work-balanced partitions.

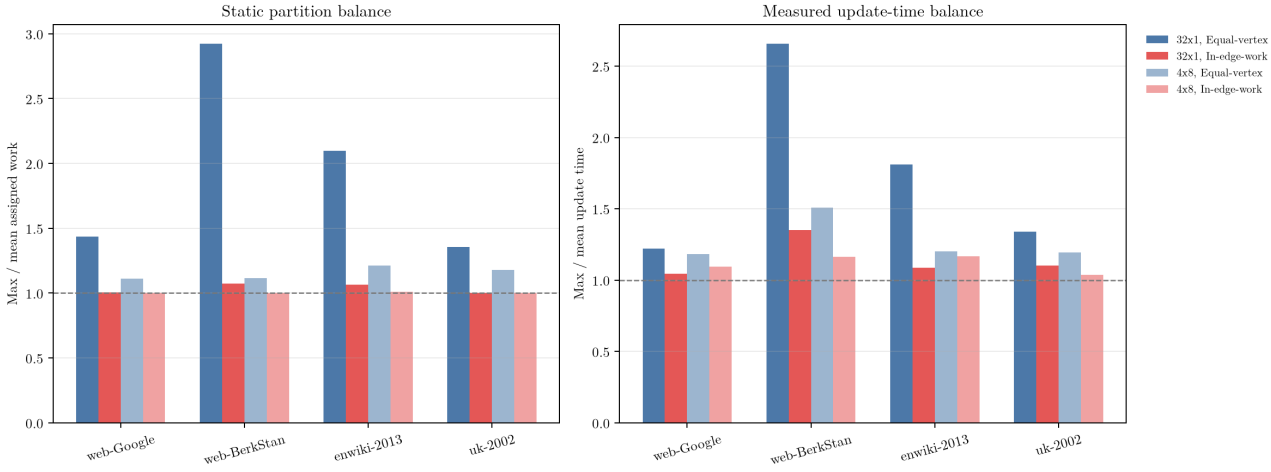


Figure 6: Static and measured imbalance by partition.

5.6 Experiment 6: Phase timing and the scaling limit

The phase experiment contains 160 measured runs at 4, 32, 64, and 128 CPUs. Communication becomes dominant as CPU count increases. For MPI-only, the median communication share from 4 to 128 CPUs rises from 20.8% to 96.6% on Google, 34.5% to 99.4% on BerkStan, 23.4% to 93.0% on enwiki, and 30.3% to 94.5% on UK-2002.

Hybrid execution reduces communication pressure by using fewer ranks. At 128 CPUs, communication accounts for 75.9% of enwiki hybrid time and 86.0% of UK hybrid time, compared with 93.0% and 94.5% for MPI-only. `Allgatherv` is the largest 128-CPU component in most configurations: 63.5%-80.4% for MPI-only and 38.9%-75.3% for hybrid. enwiki hybrid retains the largest useful compute share at 128 CPUs, 24.1%.

These measurements explain both the early saturation of small graphs and the advantage of Hybrid execution. In-edge-work partitioning reduces waiting caused by unequal local computation, while the full-vector collective remains the main opportunity for further improvement. The phase results therefore

motivate the demand-driven communication design.

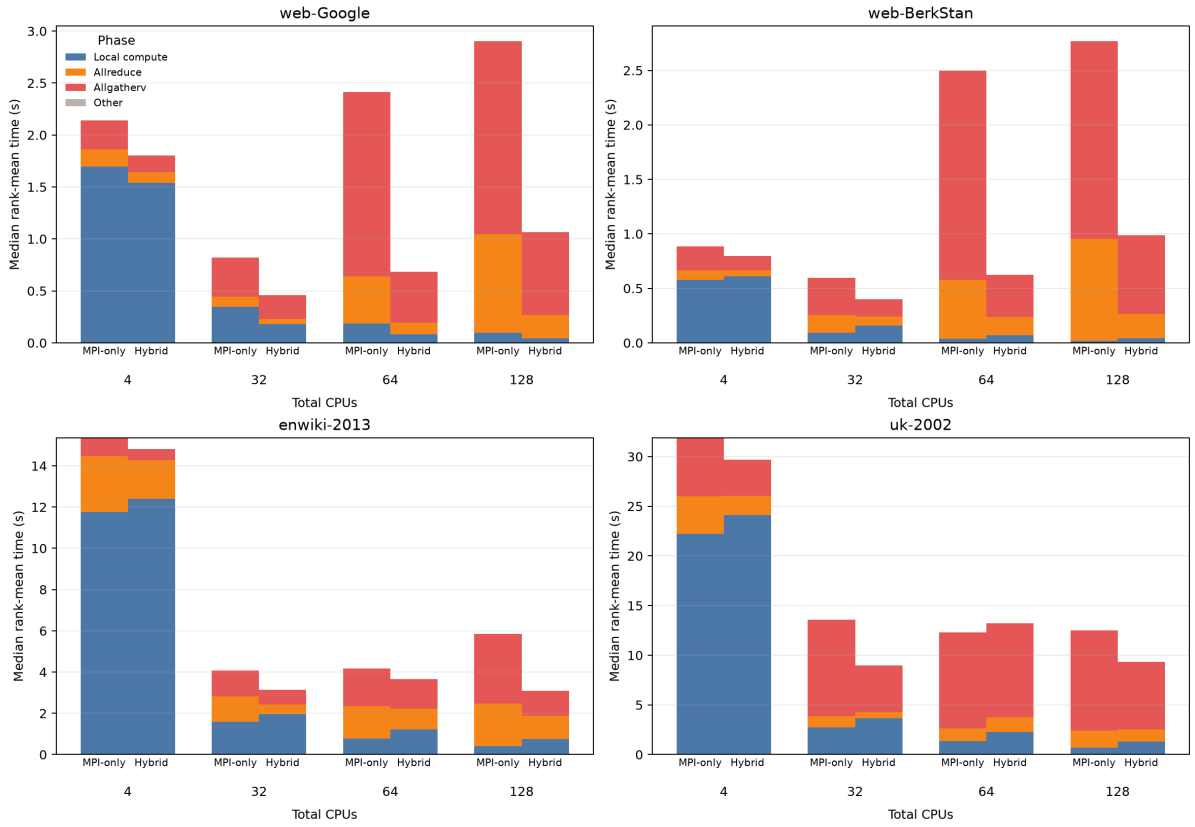


Figure 7: Phase-time breakdown by graph and configuration.

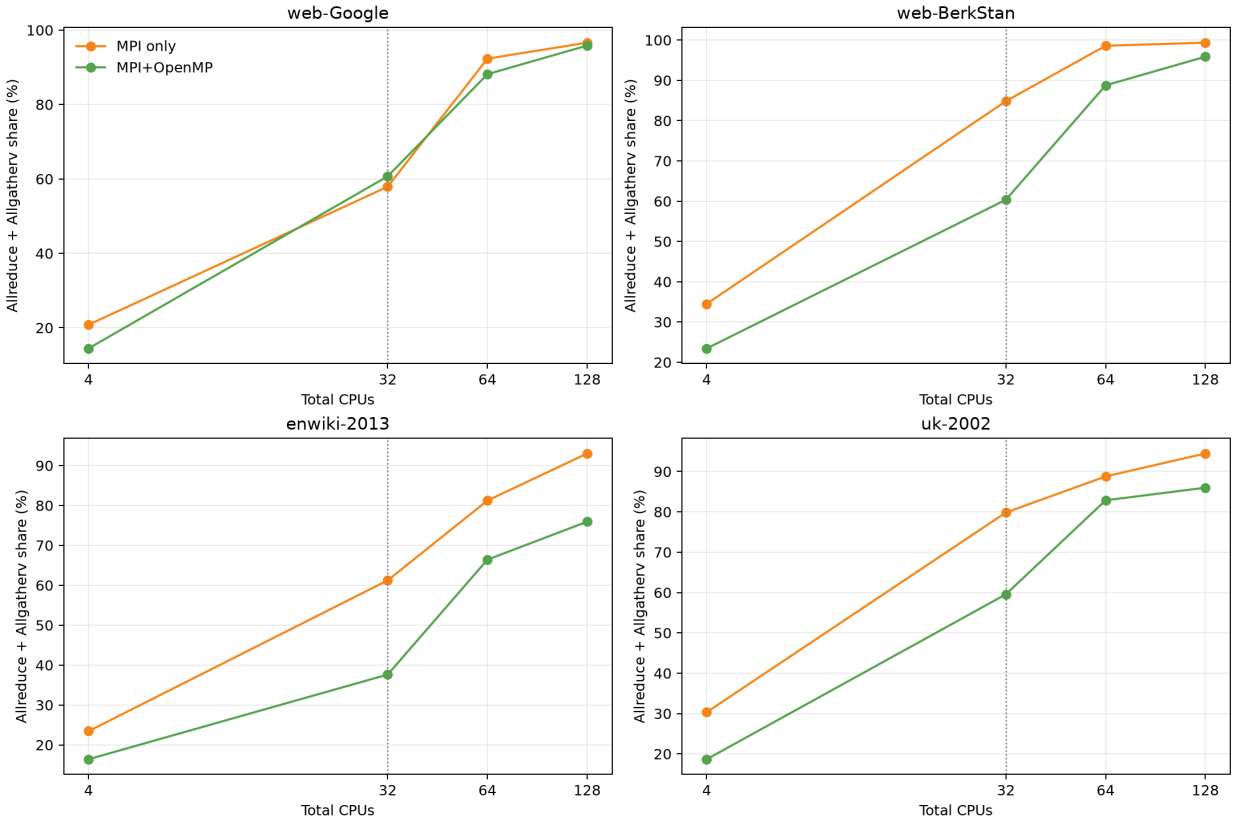


Figure 8: Collective-communication share by graph and configuration.

5.7 Experiment 7: Demand-driven score exchange

The controlled exchange experiment contains 160 measured runs: four graphs, two CPU counts, two rank/thread layouts, two exchange methods, and five repetitions. All runs converged. The table reports median full-vector time divided by median demand-driven time and the corresponding reduction in per-iteration score payload.

Table 7: Demand-driven speedup and score-payload reduction.

Data set	CPUs	MPI speedup	Hybrid speedup	MPI payload reduction	Hybrid payload reduction
web-Google	32	2.140x	1.126x	6.54x	1.73x
web-Google	64	2.707x	2.127x	12.04x	2.41x
web-BerkStan	32	2.997x	1.595x	31.54x	12.33x
web-BerkStan	64	2.234x	2.820x	44.68x	17.69x
enwiki-2013	32	1.289x	0.850x	6.17x	1.62x
enwiki-2013	64	1.359x	1.121x	10.64x	2.34x
uk-2002	32	3.310x	1.833x	105.50x	15.82x
uk-2002	64	4.983x	5.260x	202.70x	29.44x

Demand-driven exchange is faster in 15 of 16 comparisons. Its largest gain is 5.260x for UK Hybrid at 64 CPUs; UK MPI at 64 CPUs reduces score payload by 202.70x and runs 4.983x faster. The enwiki Hybrid case at 32 CPUs provides a boundary condition: a 1.62x payload reduction accompanies 0.303 s of packing, and solve time changes from 2.908 s to 3.421 s. Across the experiment, performance reflects the balance between payload reduction and the costs of packing, irregular partner communication, and compact-state lookup.

Dependency construction takes 0.022-1.178 s and is excluded from solve time. Packing is included in solve time and ranges from 0.013 to 0.349 s in the reported medians. The explicit exchange-state estimate is lower for demand exchange in 13 of 16 configurations. The process peak-RSS ratio remains approximately one because rank 0's earlier complete-graph load determines its high-water mark, while the exchange-state metric describes steady iterative storage directly.

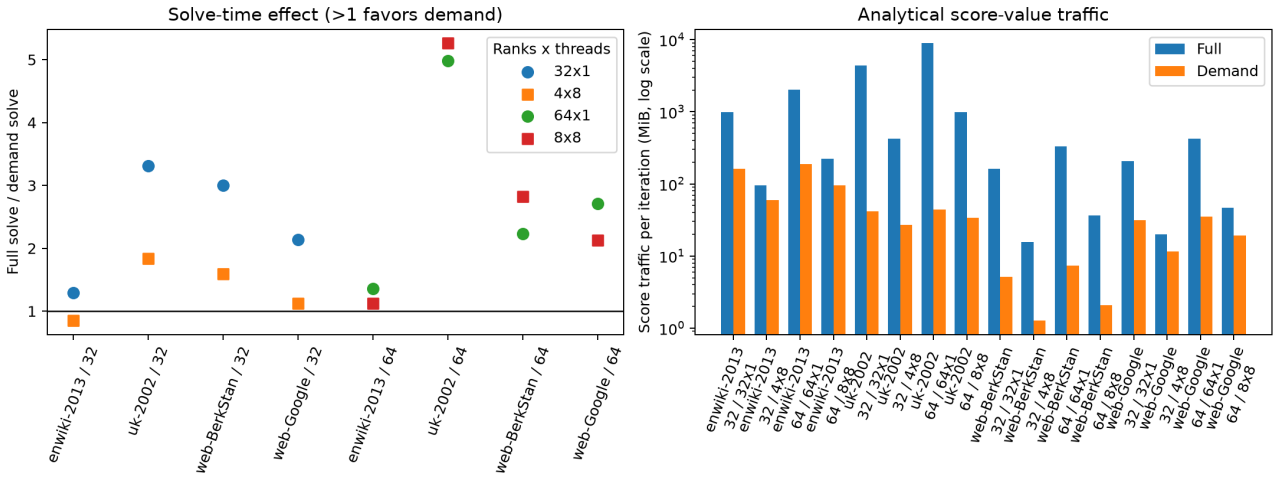


Figure 9: Demand-exchange runtime and score traffic.

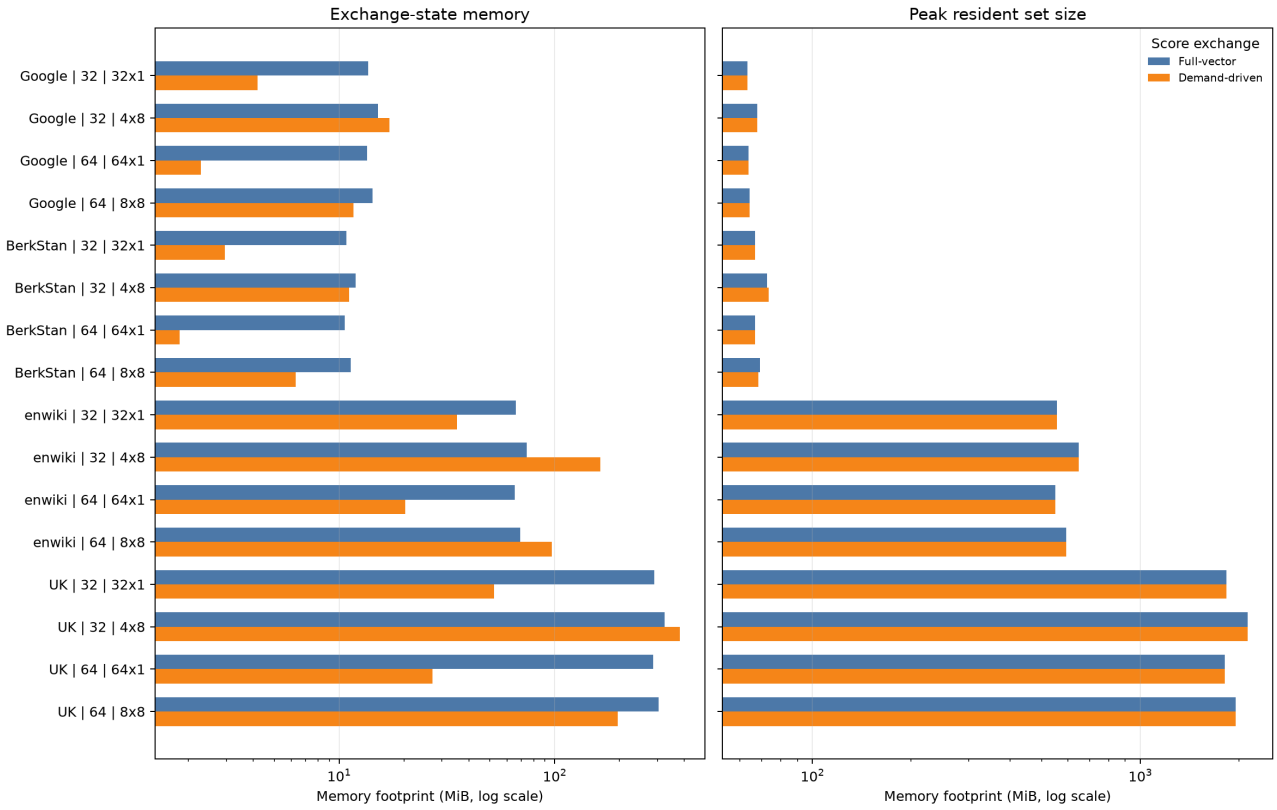


Figure 10: Exchange-state and peak-RSS memory.

6. Discussion

6.1 Effects of graph structure

Graph size alone does not determine scaling. BerkStan has fewer vertices than Google but more edges, while enwiki and UK contain substantially more work. The comprehensive layout sweep nevertheless places the full-vector minimum at 32 CPUs for all four graphs because multi-node collective cost grows faster than the remaining local work falls. The larger graphs retain stronger absolute performance at 64 and 128 CPUs, and UK selects a different 128-CPU layout from the other graphs. Experiment 5 further shows that skewed in-degree creates local waiting: BerkStan has the strongest equal-vertex static imbalance, while enwiki obtains the largest MPI-only runtime improvement from work-aware partitioning.

The induced UK samples demonstrate another structural effect. Sampling a vertex subset and retaining only internal edges changes density and convergence behavior, creating realistic nested graphs whose workload grows nonlinearly with the sampled vertex fraction. Replica graphs provide controlled work per CPU through repeated disconnected structure. The two methods offer complementary views of scaling behavior.

6.2 Design trade-offs

The full-vector baseline favors simplicity, direct score lookup, and regular collective communication. Its cost is $O(N)$ score memory per rank and one full-vector `Allgatherv` per iteration. The demand-driven variant replaces this with compact local and remote score arrays plus a fixed dependency plan. It reduces iterative payload and explicit state when remote dependency sets are sparse, but adds one-time dependency construction, per-iteration packing, edge-to-slot metadata, and potentially irregular `Alltoallv` traffic.

The measured trade-off depends on graph structure and rank count. UK has the sparsest cross-owner score demand relative to its full vector and receives the largest gains. Hybrid layouts have fewer, larger ownership ranges, so they often expose less payload reduction than MPI-only layouts. In the measured implementation, demand-score packing is a serial loop within each MPI rank, even in Hybrid mode. The

enwiki 32-CPU Hybrid case spends 0.303 s in this packing stage; together with dependency-management and irregular communication costs, this exceeds the saving from a 1.62x payload reduction. Dependency construction is excluded from solve time; repeated solves can amortize this setup, while one-shot applications include it in end-to-end cost. Peak RSS captures the earlier complete-graph loading high-water mark, and the exchange-state metric characterizes optimized steady iterative storage.

In-edge-work partitioning remains complementary: it changes ownership boundaries to reduce update imbalance, while demand exchange changes score movement. The controlled comparison applies the same in-edge-work partition to both exchange methods, so its measured differences quantify communication design under a common load-balancing policy.

6.3 End-to-end optimization pipeline

Figure 11 presents a representative load-optimization pipeline for uk-2002 under a fixed Vanda environment. The graph, numerical parameters, IN-002 node, 32-CPU allocation, warm-up, and five measured repetitions are held constant. Serial uses one core; MPI-only uses 32x1, and the Hybrid stages use the best fixed-32 layout, 2x16. Median solve time falls from 78.663 s for Serial to 13.415 s for MPI-only, 6.624 s for MPI+OpenMP, 6.449 s with in-edge-work partitioning, and 4.121 s with demand-driven exchange. The final configuration is 19.09x faster than Serial. The serial median is within 2.65% of the earlier IN-002 UK baseline of 80.802 s, and its five-run coefficient of variation is 0.14%. The floating bars visualize the incremental reduction delivered by each stage in this representative optimization scenario.

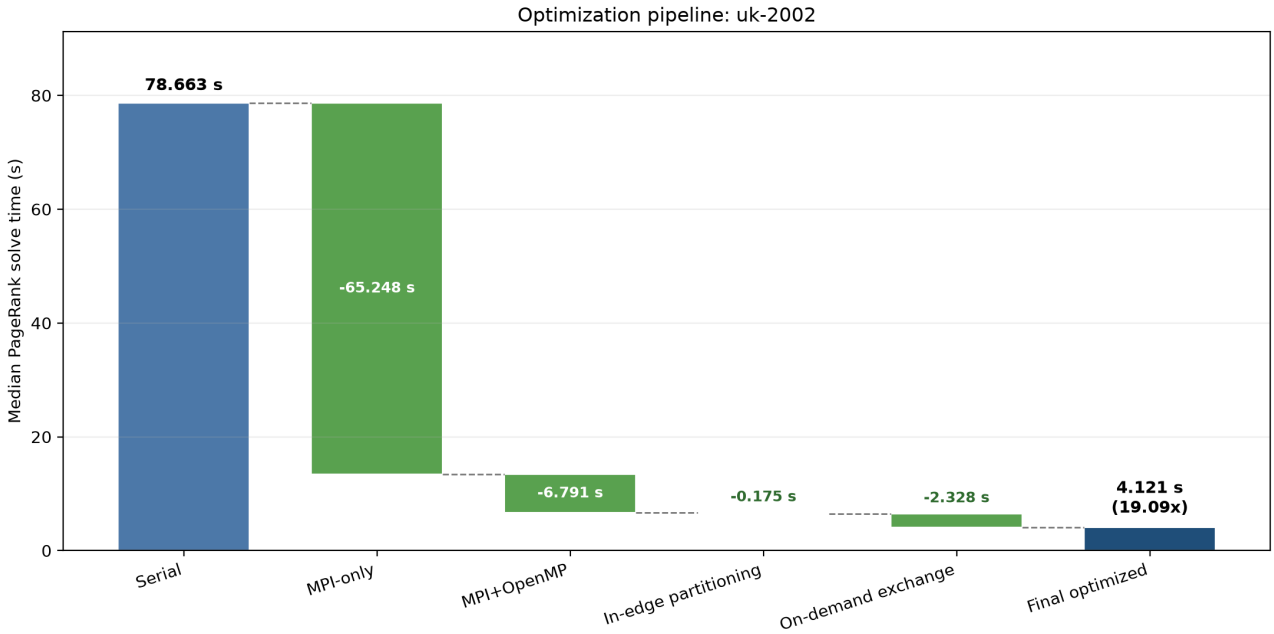


Figure 11: Optimization-pipeline waterfall on uk-2002.

6.4 Threats to validity

- Five repetitions characterize ordinary variation but do not eliminate cluster contention.
- Layouts at a given graph/resource scope share one PBS job; CPU-count scopes remain scheduler-placed, so the report uses same-sweep medians and records node identity in the CSVs.
- Serial baselines differ across historical job series. Fixed-32 decomposition therefore reports no serial speedup, and comprehensive speedups use only the Serial measurement from the same IN-015 per-graph job. Serial timing variability is also non-negligible for the smaller comprehensive cases, so the reported ratios are median-based estimates rather than hardware-invariant constants.
- Phase instrumentation is reported as a dedicated experiment with its own controlled measurements and supports interpretation of the strong-scaling trends.

- The demand payload metric counts score values and excludes MPI protocol and synchronization overhead.
- Demand-score packing is serial within each rank in the evaluated implementation; this especially limits Hybrid configurations with few ranks and many threads.
- Peak RSS records the process high-water mark established by rank 0's complete-graph loading stage; exchange-state bytes separately characterize steady iterative storage.
- The implementation uses standard MPI `int` counts, so distributed graph sizes must remain within the corresponding count limits.

7. Conclusion

This project produced a common-CSR PageRank implementation in Serial C++, MPI-only, and MPI+OpenMP forms and evaluated it on four real graphs. All 1,625 formal Vanda runs converged; deterministic test graphs establish Serial/parallel consistency, and NetworkX independently validates complete web-Google vectors from Serial and demand-driven MPI runs. The comprehensive strong-scaling sweep tests every feasible rank/thread layout at each CPU count. Hybrid execution provides the best full-vector configurations because it retains local shared-memory parallelism while reducing MPI ranks, and all four graphs reach their best measured full-vector time at 32 CPUs. The best speedups range from 6.32x to 12.20x.

The experiments identify and address two limits. Unequal incoming-edge work causes measurable waiting, and in-edge-work partitioning improves seven of eight comparisons. Full-vector score synchronization is the larger high-scale constraint: communication exceeds 93% of MPI-only time at 128 CPUs on every graph. Demand-driven exchange reduces that volume and improves 15 of 16 controlled comparisons, with gains up to 5.260x and payload reductions up to 202.70x. The enwiki 32-CPU Hybrid case establishes the boundary where a modest payload reduction is outweighed by packing and dependency overhead. Demand exchange is therefore most effective when cross-owner dependencies are sparse enough to offset its planning, metadata, and irregular-communication costs.

AI Use Declaration

We used OpenAI Codex as a development assistant. We defined the objectives and experimental scope, selected the Serial, MPI-only, and MPI+OpenMP designs, chose the data and configurations, and implemented the code in C++. Codex assisted with code review and refactoring, environment configuration, robustness testing, and report structure design. We reviewed the code, logs, validation outputs, figures, and report, and take full responsibility for the final submission.

References

[1] S. Brin and L. Page, "The Anatomy of a Large-Scale Hypertextual Web Search Engine," Computer Networks and ISDN Systems, vol. 30, no. 1-7, pp. 107-117, 1998.

[2] Message Passing Interface Forum, "MPI: A Message-Passing Interface Standard, Version 4.1," 2023. https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf

[3] OpenMP Architecture Review Board, "OpenMP Application Programming Interface, Version 5.2," 2021. https://www.openmp.org/spec-html/5.2/openmp.html

[4] NetworkX Developers, "pagerank - NetworkX 3.7 documentation," 2026. https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.link_analysis.pagerank_alg.pagerank.html

[5] J. Leskovec and A. Krevl, "SNAP Datasets: Stanford Large Network Dataset Collection," 2014. https://snap.stanford.edu/data/

[6] SNAP, "Web graph and Wikipedia hyperlink data sets." https://snap.stanford.edu/data/

[7] Laboratory for Web Algorithmics, "UK-2002 Web Graph." https://law.di.unimi.it/webdata/uk-2002/